

Python

Summary Session 19



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Большие объёмы данных



Когда программы становятся больше, появляется необходимость хранить и обрабатывать большие объёмы данных — о товарах, пользователях, сообщениях, заказах и т.д.

Экспресс-опрос

?

Назовите плюсы использования баз данных

Использование баз данных позволяет:

-  Хранить структурированные данные вне программы
-  Быстро искать, обновлять и удалять записи
-  Работать с очень большими объёмами данных без загрузки их всех в память
-  Обеспечить многопользовательский доступ и защиту данных
-  Делать программы более надёжными и масштабируемыми



Подключение к базе данных

Работа в Python с MySQL

Нужно использовать специальную библиотеку, которая умеет:



устанавливать соединение с сервером базы данных



отправлять SQL-запросы и получать результаты



обрабатывать ошибки при работе с базой

Экспресс-опрос

?

Какие популярные библиотеки для работы с базами данных вы помните?

Популярные библиотеки для работы с MySQL

Библиотека	Особенности
PyMySQL	Простая, полностью на Python
mysql-connector-python	Официальная библиотека от MySQL, не всегда удобно интегрируется
MySQLdb (mysqlclient)	Очень быстрая, но сложнее в установке (особенно на Windows)

Установка библиотеки PyMySQL

```
pip install pymysql
```

В этой лекции мы будем использовать PyMySQL, потому что она работает стабильно на всех операционных системах

Создание соединения

Чтобы подключиться к серверу MySQL, нужно:

- Импортировать модуль `pymysql`,
- Вызвать функцию `pymysql.connect()`,
- Передать в неё параметры подключения.

Синтаксис

```
import pymysql                # импорт модуля

connection = pymysql.connect(
    host="localhost",         # адрес сервера базы данных
    user="root",              # имя пользователя
    password="yourpassword",  # пароль для указанного пользователя
    database="yourdatabase",  # название базы данных (необязательно)
    charset="utf8mb4",        # кодировка соединения (необязательно, указывать для поддержки русского)
)
```

Создание соединения

Особенности:

- Параметры подключения можно передавать **вручную**.
- Либо можно **хранить параметры в словаре** и **распаковать его** с помощью ****** при подключении.

Пример

```
import pymysql

config = {'host': 'ich-db.edu.itcareerhub.de',
          'user': 'ich1',
          'password': 'password',
          'database': 'hr',
          }

connection = pymysql.connect(**config) # распаковка словаря как аргументы
```

Экспресс-опрос

?

В каких случаях удобно использовать словарь?

Когда удобно использовать словарь



Когда параметры нужно передавать в разных частях программы.



Когда подключений будет много — например, к разным базам данных.



Когда параметры удобно хранить в файлах (.env, JSON, YAML и т.д.).

Проверка соединения

```
if connection.open:  
    print("Connection successful!")
```

Заккрытие соединения

```
if connection.open:  
    print("Connection successful!")
```

Почему важно закрывать соединение

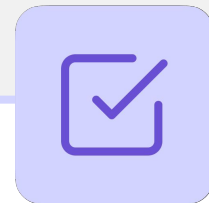
- Открытые соединения занимают ресурсы сервера базы данных.
- Если не закрывать соединения, можно быстро исчерпать лимит подключений.
- Правильная практика: **всегда закрывать соединение**, как только оно больше не нужно.

Экспресс-опрос

?

Зачем нужен объект курсора?

Работа с курсором



После подключения к базе данных необходимо создать **объект курсора**, через который:

- отправляются SQL-запросы в базу,
- получаются результаты выполнения запросов.

Создание курсора



Синтаксис

```
cursor = connection.cursor()
```

Пояснения

- `cursor()` — метод соединения (`connection`), который создаёт новый курсор для работы.
- Один `cursor` используется для выполнения одного или нескольких запросов.

Заккрытие курсора



Синтаксис

```
cursor.close()
```

Пояснения

- После использования курсор нужно **закрыть вручную**, чтобы освободить ресурсы
- После закрытия курсор становится недействительным и для новых запросов нужно создать новый курсор

Выполнение запросов



Пример

```
cursor = connection.cursor()  
cursor.execute("SELECT * FROM departments")
```

Пояснения

- После создания курсора можно **выполнять SQL-запросы** с помощью метода `execute()`

Особенности работы с курсором

-  `cursor.execute(sql)` — отправляет SQL-запрос к базе данных.
-  SQL-запрос передаётся в виде строки.
-  После выполнения запроса можно получить результат.
-  Можно выполнять любой SQL-запрос: `SELECT`, `INSERT`, `UPDATE`, `DELETE` и др.
-  После каждого нового запроса старые результаты очищаются (если результат не считать, некоторые библиотеки, например `mysql-connector`, вызовут ошибку при следующем запросе).
-  При ошибке в запросе выбрасывается исключение.

Выполнение запросов



Пример

```
cursor.execute("SELECT * FROM employees")  
  
for row in cursor:  
    print(row)
```

Пояснения

- После выполнения запроса SELECT данные остаются в курсоре, пока не будут получены. Для их получения курсор можно использовать **как итератор** в цикле
- Каждая итерация даёт **одну строку результата** (в виде кортежа)
- Все строки читаются **последовательно**, до окончания выборки

Экспресс-опрос

A small purple square icon containing a white question mark.

Почему при выполнении запросов в базу данных никогда нельзя подставлять значения напрямую в строку SQL?

Важно



При выполнении запросов в базу данных **никогда нельзя подставлять значения напрямую в строку SQL**. Это может привести к **SQL-инъекциям**.



SQL-инъекция

Это тип уязвимости, при котором злоумышленник может вставить произвольный SQL-код в запрос, чтобы получить несанкционированный доступ к данным, изменить или удалить информацию в базе данных

Пример

```
# Прямое включение пользовательского ввода
user_input = "1 OR 1=1"
sql = f"SELECT * FROM employees WHERE employee_id = {user_input}"
cursor.execute(sql) # выполнит: SELECT * FROM employees WHERE employee_id = 1 OR 1=1
# Получены все записи, вместо одной
for r in cursor:
    print(r)
```

Использование параметризованных запросов

```
cursor.execute(
    "SELECT * FROM table_name WHERE column1 = %s AND column2 > %s",
    (value1, value2)
)
```

- Первый аргумент — SQL-запрос с плейсхолдерами %s
- Второй аргумент — **последовательность значений** (кортеж или список), которые подставляются вместо %s

Пример: несколько значений

```
cursor.execute(
    "SELECT * FROM employees WHERE department_id = %s OR salary > %s",
    (60, 20000)
)
for r in cursor:
    print(r)
```

- Подставляются оба значения в соответствующие %s.
- Порядок значений в последовательности должен совпадать с порядком %s в запросе.

Пример: одно значение

```
cursor.execute(
    "SELECT * FROM employees WHERE department_id = %s",
    (100,) # Обязательно запятая!
)
for r in cursor:
    print(r)
```

- Если передаётся только одно значение, нужно ставить запятую после него, чтобы создать кортеж.

Экспресс-опрос

?

Назовите преимущества параметризованных запросов

Преимущества параметризованных запросов



Безопасность — защита от SQL-инъекций



Универсальность — одно и то же выражение можно выполнять с разными данными



Автоматическое экранирование — библиотека сама правильно обрабатывает типы данных (строки, числа и т.д.)



Улучшение производительности — сервер базы данных может кэшировать план выполнения запроса

Именованные параметры



Вместо позиционных %s, в запросах можно использовать **именованные параметры**. Это делает код **более читабельным**, особенно когда передаётся много значений.

Именованные параметры

```
cursor.execute(
    "SELECT * FROM table_name WHERE column1 = %(param1)s AND column2 > %(param2)s",
    {"param1": value1, "param2": value2}
)
```

- В SQL-запросе используются плейсхолдеры вида %(name)s
- Вторым аргументом передаётся словарь, где ключи соответствуют плейсхолдерам
- Порядок ключей в словаре не важен — главное, чтобы ключи словаря совпадали с именами плейсхолдеров в запросе

Пример

```

cursor.execute(
    "SELECT * FROM employees WHERE department_id = %(dep_id)s OR salary > %(min_salary)s",
    {"min_salary": 20000, "dep_id": 60}
)
for r in cursor:
    print(r)
  
```

Экспресс-опрос

?

Назовите преимущества именованных параметров

Преимущества



Повышается читаемость запроса



Удобно, когда значения приходят как словарь



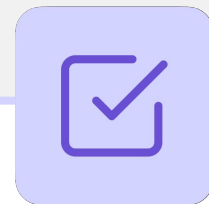
Легче избежать ошибок с порядком аргументов

Экспресс-опрос

?

Для чего используется конструкция `with`?

Важно



Чтобы упростить работу с соединением и курсором, можно использовать конструкцию `with`, которая **автоматически закрывает** ресурсы даже при ошибках.

Это делает код **чище** и **защищает от утечек соединений**.

Пример

```
with pymysql.connect(
    host='ich-db.edu.itcareerhub.de',
    user='ich1',
    password='password',
    database='hr'
) as connection: # автоматически закрывает connection
    with connection.cursor() as cursor: # автоматически закрывает cursor
        cursor.execute("SELECT * FROM employees")
        for row in cursor:
            print(row)
```

Особенности



Блок `with` автоматически вызывает `close()` для курсора и/или соединения



Код становится компактнее и безопаснее



Даже если внутри блока произойдёт ошибка — ресурсы **корректно закроются**

Экспресс-опрос

?

Какой метод позволяет окончательно записать изменения в базу данных?

Метод commit



Пояснения

Чтобы окончательно записать изменения в базу данных, необходимо вызвать метод `commit()`

Синтаксис

```
connection.commit()
```

Причины вызова commit



Без `commit()` изменения **не сохраняются** и будут **потеряны** при закрытии соединения



Это поведение обеспечивает **безопасность**: изменения применяются только после явного подтверждения



Создание базы данных и таблиц (`CREATE DATABASE`, `CREATE TABLE`) **не требует вызова** `commit()` — эти команды сохраняются сразу.

Подключение к серверу с правами на изменения



Пояснения

- Чтобы создавать базы и таблицы, подключитесь к серверу с правами на изменения
- Обратите внимание, что базу данных при подключении можно не указывать и выбрать ее позже

Код

```
config = {'host': 'ich-edit.edu.itcareerhub.de',
          'user': 'ich1',
          'password': 'ich1_password_ilovedbs',
          }

connection = pymysql.connect(**config)
```

Пример: создание базы и таблицы sales

```
with connection.cursor() as cursor:
    # Создание новой базы данных (если ещё не существует)
    cursor.execute("CREATE DATABASE IF NOT EXISTS market")
    cursor.execute("USE market")

    # Создание таблицы продаж
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS sales (
            id INT AUTO_INCREMENT PRIMARY KEY,
            item_name VARCHAR(100),
            quantity INT,
            price DECIMAL(10, 2),
            sale_date DATE
        )
    """)
```


Пример: вставка данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "INSERT INTO sales (item_name, quantity, price, sale_date) VALUES (%s, %s, %s,  
        %s)",  
        ("Keyboard", 2, 45.50, "2024-06-15")  
    )  
    connection.commit() # Сохраняем изменения
```

Пример: обновление данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "UPDATE sales SET quantity = %s WHERE item_name = %s",  
        (3, "Keyboard")  
    )  
    connection.commit()
```

Пример: удаление данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "DELETE FROM sales WHERE item_name = %s",  
        ("Keyboard",)  
    )  
    connection.commit()
```

Экспресс-опрос

?

Какой метод используется для многократного выполнения одного и того же SQL-запроса с разными данными?

Метод executemany



Метод `executemany()` используется для **многократного выполнения одного и того же SQL-запроса** с разными данными. Это удобно и эффективно при массовом добавлении, обновлении или удалении строк.

Пример

```
with connection.cursor() as cursor:
    cursor.executemany(
        "INSERT INTO sales (item_name, quantity, price, sale_date) VALUES (%s, %s, %s, %s)",
        [
            ("Notebook", 3, 19.99, "2024-06-15"),
            ("Pen", 10, 1.99, "2024-06-16"),
            ("Bag", 1, 49.90, "2024-06-17")
        ]
    )
    connection.commit()
```

Особенности



Работает только с **изменяющими запросами** (INSERT, UPDATE, DELETE)



Вторым аргументом передаётся **список кортежей**



Экономит ресурсы, т.к. запрос компилируется один раз, а выполняется много

Экспресс-опрос



Что такое MongoDB?



MongoDB

Это документоориентированная база данных, с которой можно удобно работать и из Python

Библиотека pymongo



Пояснения

Для работы с MongoDB в Python используется библиотека pymongo, которая позволяет выполнять запросы к базе в привычной форме: через словари, как в консоли MongoDB

Установка

```
pip install pymongo
```

Подключение к серверу



Для работы с базой данных MongoDB необходимо создать **объект** MongoClient, передав в него строку подключения. Она содержит все параметры для подключения: логин, пароль, адрес сервера и настройки авторизации.

Подключение к серверу

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://ich_editor:verystrongpassword"
    "@mongo.itcareerhub.de/?readPreference=primary"
    "&ssl=false&authMechanism=DEFAULT&authSource=ich_edit"
)
```

Основные параметры:

- `ich_editor` — имя пользователя
- `verystrongpassword` — пароль
- `mongo.itcareerhub.de` — адрес сервера
- `authSource=ich_edit` — имя базы, в которой хранятся учётные данные

Проверка подключения

```
client.admin.command("ping")  
print("Connection successful!")
```

Выбор базы данных



Пример

```
db = client["ich_edit"]
```

Особенности

- Если база не существует, она будет создана при первом добавлении данных
- Подключение устанавливается лениво — реально соединение происходит при первом запросе

Выбор коллекции



Пример

```
products = db["products"]
```

Особенности

Коллекции тоже создаются автоматически при первом добавлении данных

Заккрытие соединения



Пример

```
client.close()
```

Особенности

MongoClient сам закрывает соединение при завершении программы, но при необходимости его можно закрыть вручную

Экспресс-опрос

?

В каком виде все данные хранятся в MongoDB?

Добавление данных



В MongoDB все данные хранятся в **документах**, которые представляют собой обычные Python-словари (dict). MongoDB позволяет добавлять документы несколькими способами

1. Добавление одного документа — insert_one

```
product = {
    "name": "Notebook",
    "price": 5.99,
    "stock": 120
}

result = products.insert_one(product)
print("Inserted ID:", result.inserted_id)
```

- Метод `insert_one()` вставляет **один документ**.
- Возвращает объект `InsertOneResult`, из которого можно получить `_id` добавленного документа.

2. Добавление нескольких документов — insert_many

```
items = [
    {"name": "Pen", "price": 1.50, "stock": 300},
    {"name": "Pencil", "price": 0.99, "stock": 500},
    {"name": "Eraser", "price": 0.75, "stock": 200},
]

result = products.insert_many(items)
print("Inserted IDs:", result.inserted_ids)
```

- Метод `insert_many()` принимает список словарей и добавляет их сразу.
- Возвращает `InsertManyResult` с перечнем `_id` всех вставленных документов.

Особенности



Если поле `_id` не указано вручную, MongoDB создаёт его автоматически.



Вы можете вставить любой словарь, если структура документов в коллекции не фиксирована.



В отличие от SQL, необязательно заранее описывать структуру коллекции.

Экспресс-опрос

?

Какие методы используются для получения документов из коллекции?

Чтение данных



Для получения документов из коллекции используются методы `find_one()` и `find()`

Пример метода find_one()

```
# Получить один документ  
doc = products.find_one()  
print(doc)
```


Метод find_one()

Особенности:

- Метод `find_one()` возвращает **первый попавшийся** документ (или `None`, если ничего не найдено).
- Можно передавать **условие поиска** — словарь с нужными параметрами

Пример

```
doc = products.find_one({"price": {"$lt": 5}})
print(doc)
```

Экспресс-опрос

?

Что позволяет не загружать сразу все документы в память, а перебирать их по одному — как итератор?

Работа с курсором



Метод `find()` возвращает **объект-курсор**. Это позволяет **не загружать сразу все документы в память**, а перебирать их по одному — как итератор.

Примеры метода find()

Получить все документы

```
docs = products.find()
print(docs)
for item in docs:
    print(item)
```

Получить все документы с условием

```
cursor = products.find({"price": {"$gt": 1}})

for doc in cursor:
    print(doc)
```

Курсор

Особенности:

- Курсор можно использовать в `for`, как обычный итератор.
- После окончания перебора курсор становится **пустым**.
- Можно настроить **лимит, сортировку и пропуск** через методы `.limit()`, `.sort()`, `.skip()`

Пример

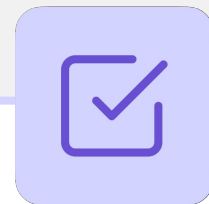
```
for doc in products.find().sort("price", -1).skip(1).limit(2):  
    print(doc)
```

Экспресс-опрос

?

Что такое проекция полей?

Проекция полей



По умолчанию `find()` и `find_one()` возвращают **все поля** документа.

Можно указать, **какие поля нужны**, — это называется **проекцией**.

Важно помнить:

- Первый параметр `find()` – словарь с условиями поиска
- Второй параметр `find()` – ограничения проекции
- Если условий поиска нет, то первый параметр нужно передавать пустым словарем

Примеры

```
# Вернёт только name, price и id (по умолчанию)
for doc in products.find({}, {"name": 1, "price": 1}):
    print(doc)

# Выбрать продукты с ценой больше 100 и исключить _id:
for doc in products.find({"price": {"$gt": 100}}, {"_id": 0}):
    print(doc)

# Оставить только name (все остальные исключаются, включая _id)
for doc in products.find({}, {"_id": 0, "name": 1}):
    print(doc)
```


Экспресс-опрос

?

Какие методы используются для обновления данных?

Обновление данных



Чтобы изменить документ в MongoDB, используются методы `update_one()` и `update_many()`. Они принимают **фильтр** (поиск нужного документа) и **модификатор** (`$set`, `$inc`, и др.).

Пример: изменить цену одного товара

```
result = products.update_one(
    {"name": "Notebook"},      # фильтр
    {"$set": {"price": 24.99}} # изменение
)

print("Matched:", result.matched_count)
print("Modified:", result.modified_count)
```

- Метод `update_one()` изменяет **только первый подходящий** документ.
- Модификатор `$set` обновляет указанное поле.
- `matched_count` — сколько документов подошли под условие.
- `modified_count` — сколько документов реально изменились.

Пример: увеличить цену всех товаров

```
result = products.update_many(
    {}, # пустой фильтр – все документы
    {"$inc": {"price": 1}} # увеличить поле price на 1
)

print("Matched:", result.matched_count)
print("Modified:", result.modified_count)
```

- Метод `update_many()` изменяет **все подходящие** документы.
- `$inc` используется для **увеличения числового значения**.
- Возвращаемый объект также содержит `matched_count` и `modified_count`.

Удаление данных



MongoDB позволяет удалять как один документ, так и все, подходящие под условие.

Пример: удалить один документ

```
result = products.delete_one({"name": "Notebook"})  
print("Deleted:", result.deleted_count)
```

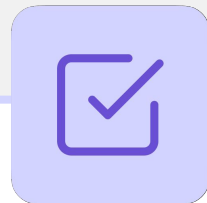
- `delete_one()` удаляет **первый** документ, подходящий под фильтр
- `deleted_count` покажет, сколько документов было удалено (0 или 1)

Пример: удалить все товары по фильтру

```
result = products.delete_many({"price": {"$lt": 2}})
print("Deleted:", result.deleted_count)
```

- `delete_many()` удаляет **все документы**, удовлетворяющие условию

Обработка ошибок



При работе с базой MongoDB через pymongo могут возникать исключения, например, при проблемах с подключением или ошибках в запросах. Чтобы программа не завершалась с ошибкой, нужно использовать блоки `try / except`.

Пример

```
from pymongo import MongoClient, errors

try:
    client = MongoClient(
        "mongodb://ich_editor:wrong_pass@mongo.itcareerhub.de/?authSource=ich_edit"
    )
    db = client["store"]
    products = db["products"]
    products.insert_one({"name": "Lamp", "price": 15.99})
except errors.ConnectionFailure:
    print("Ошибка подключения к MongoDB")
except errors.OperationFailure:
    print("Ошибка авторизации или запроса")
```

Экспресс-опрос



Назовите основные типы исключений, которые могут возникать при работе с MongoDB

Основные типы исключений

Исключение	Описание
<code>pymongo.errors.ConnectionFailure</code>	ошибка подключения к серверу MongoDB
<code>pymongo.errors.OperationFailure</code>	ошибка выполнения запроса (например, неправильные права)
<code>pymongo.errors.DuplicateKeyError</code>	попытка вставить документ с уже существующим <code>_id</code>
<code>pymongo.errors.PyMongoError</code>	базовый класс для всех исключений pymongo

Экспресс-опрос



Что такое модуль?



Модуль

Это любой `.py` файл, содержащий переменные, функции, классы и другие конструкции

Преимущества модулей



Упрощают структуру программы и позволяют разделять код по смыслу



Повышают читаемость и удобство сопровождения кода



Облегчают повторное использование и помогают избегать дублирования



Позволяют использовать готовые решения из стандартной библиотеки Python



Упрощают тестирование

Экспресс-опрос

?

Какие виды модулей вы помните?

Виды модулей

Встроенные модули

Входят в стандартную библиотеку Python (например, `math`, `random`, `datetime`)

Сторонние модули

Устанавливаются через менеджер пакетов `pip`

Пользовательские модули

Это любые `.py`-файлы, написанные самостоятельно

Работа с собственным модулем

Чтобы импортировать свой модуль, он должен находиться в той же папке или в доступном пути.

Создадим файл `math_utils.py` со следующим содержимым:

```
print("Inside math_utils.py")

def average(numbers):
    return sum(numbers) / len(numbers)

def maximum(numbers):
    return max(numbers)
```

Работа с собственным модулем

Теперь в другом файле (в той же папке), например `main.py`, можно использовать этот модуль.

```
import math_utils
from math_utils import maximum

values = [10, 20, 30]

print(math_utils.average(values))
print(maximum(values))
```

Прямой запуск модуля

Иногда модуль может использоваться и как **библиотека**, и как самостоятельный **скрипт**, запускаемый напрямую. Чтобы различать эти случаи, в Python используют конструкцию:

```
if __name__ == "__main__":  
    # Код, который будет выполняться только при прямом запуске
```

Прямой запуск модуля



Когда модуль запускается напрямую, переменная `__name__` получает значение `"__main__"`.

Если модуль **импортируется** в другой файл — `__name__` будет равно имени модуля.

Пример

Файл `math_utils.py`:

`# Это сообщение будет выведено при прямом запуске или при импорте`

```
print("Inside math_utils.py")
```

```
def average(numbers):
```

```
    return sum(numbers) / len(numbers)
```

```
def maximum(numbers):
```

```
    return max(numbers)
```

`# Это сообщение будет выведено только при прямом запуске`

```
if __name__ == "__main__":
```

```
    print("Inside main file!")
```

Только если запустить `math_utils.py` напрямую — будет выведено "Inside main file!".

Экспресс-опрос

?

Что происходит при импорте модуля в Python?

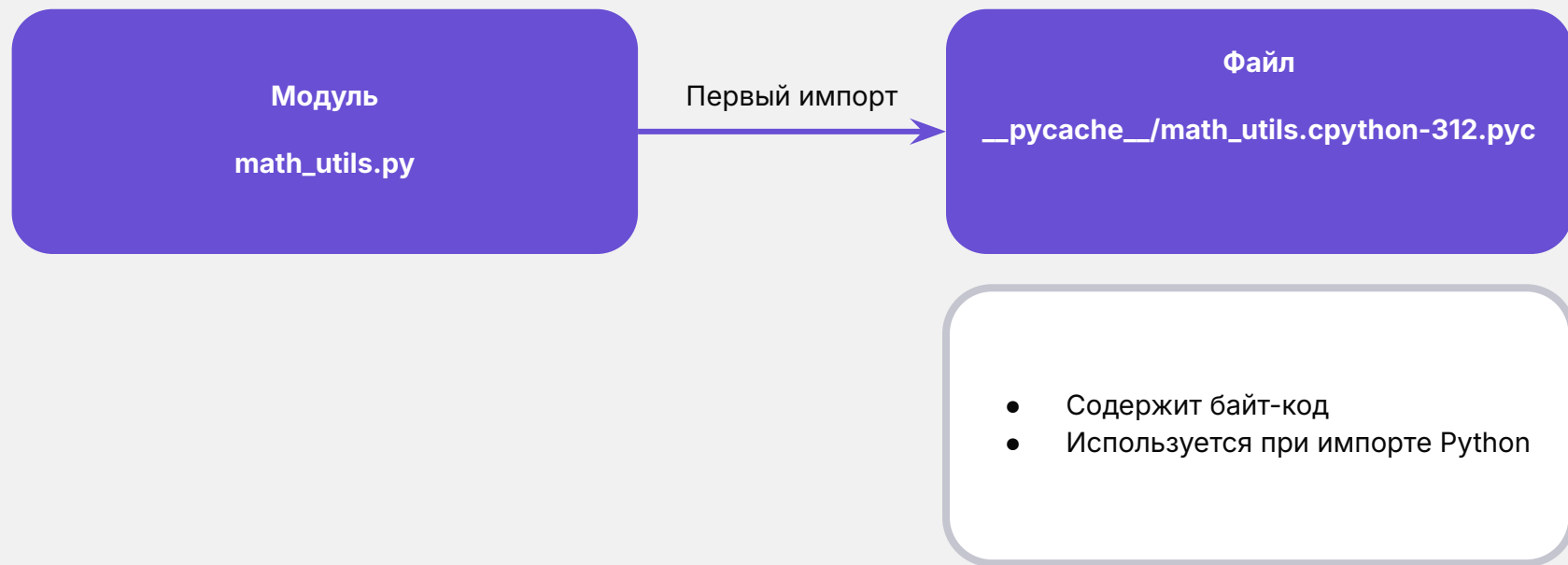
Компиляция модулей



Когда Python выполняет импорт модуля, он автоматически компилирует его в байт-код — промежуточный формат, который быстрее загружается при следующем запуске.

Скомпилированный файл сохраняется с расширением `.pyc` и помещается в папку `__pycache__`.

Пример



.рус файл пересоздается



При первом импорте модуля



Если содержимое .ру-файла изменилось



Если обновилась версия Python



ВОПРОСЫ





ЗАДАНИЕ



Выполните задание

1.1 Сопоставьте понятие с описанием:

- | | |
|--------------------------------------|---|
| 1. <code>pymysql.connect(...)</code> | a. Закрывает соединение с базой данных |
| 2. <code>connection.cursor()</code> | b. Создаёт объект для отправки SQL-запросов |
| 3. <code>cursor.execute(...)</code> | c. Подключает Python к базе данных |
| 4. <code>connection.close()</code> | d. Выполняет SQL-запрос |

Выполните задание

1.1 Сопоставьте понятие с описанием:

1. `pymysql.connect(...)`
2. `connection.cursor()`
3. `cursor.execute(...)`
4. `connection.close()`

- a. Закрывает соединение с базой данных
- b. Создаёт объект для отправки SQL-запросов
- c. Подключает Python к базе данных
- d. Выполняет SQL-запрос

Ответ: 1-с, 2-b, 3-d, 4-a



Выполните задание

1.2 Найдите правильный порядок действий для подключения к базе и выполнения запроса:

- a. Создать курсор
- b. Подключиться к базе
- c. Выполнить запрос
- d. Закрыть соединение
- e. Закрыть курсор



Выполните задание

1.2 Найдите правильный порядок действий для подключения к базе и выполнения запроса:

- a. Создать курсор
- b. Подключиться к базе
- c. Выполнить запрос
- d. Закрыть соединение
- e. Закрыть курсор

Ответ: b → a → c → e → d



Выберите верный вариант ответа

1.3 Какие параметры можно использовать в запросах `cursor.execute()`?

- a. Только позиционные %s
- b. Только именованные %(name)s
- c. И позиционные, и именованные
- d. Только строковые



Выберите верный вариант ответа

1.3 Какие параметры можно использовать в запросах `cursor.execute()`?

- a. Только позиционные %s
- b. Только именованные %(name)s
- c. И позиционные, и именованные
- d. Только строковые



Выберите верный вариант ответа

1.4 Что делает метод `commit()`?

- a. Закрывает соединение
- b. Сохраняет изменения в базе
- c. Сохраняет изменения в базе и закрывает соединение
- d. Откатывает изменения



Выберите верный вариант ответа

1.4 Что делает метод commit()?

- a. Закрывает соединение
- b. Сохраняет изменения в базе**
- c. Сохраняет изменения в базе и закрывает соединение
- d. Откатывает изменения



ВОПРОСЫ





Выберите верный вариант ответа

2.1 Что делает проекция в методе find()?

- a. Отфильтровывает документы по условиям
- b. Определяет, какие поля нужно вернуть
- c. Удаляет поля из коллекции
- d. Заменяет значения полей



Выберите верный вариант ответа

2.1 Что делает проекция в методе find()?

- a. Отфильтровывает документы по условиям
- b. Определяет, какие поля нужно вернуть**
- c. Удаляет поля из коллекции
- d. Заменяет значения полей



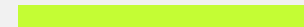
Ответьте на вопрос



2.2 Что делает \$inc в методе обновления?

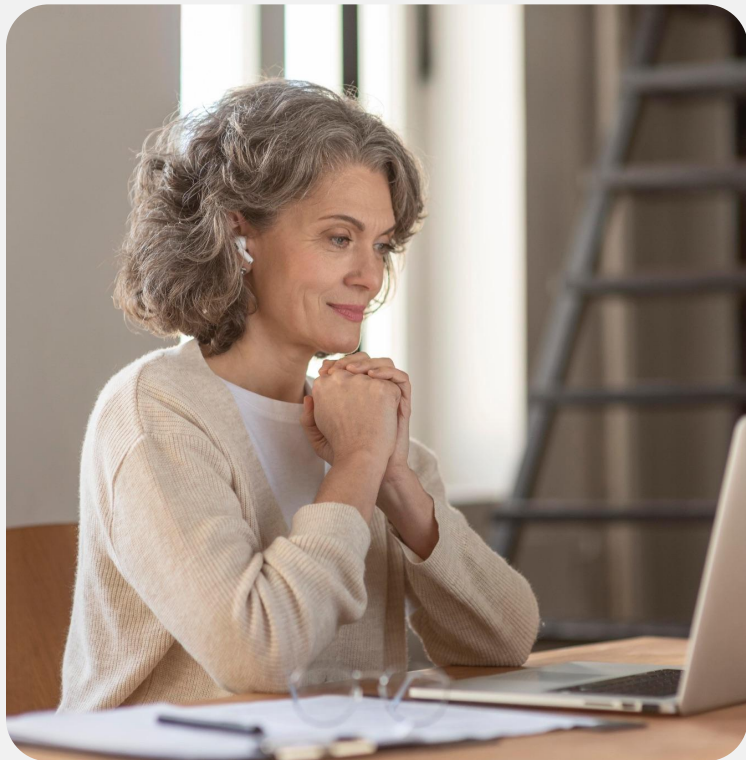


Ответьте на вопрос

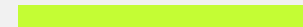


2.2 Что делает \$inc в методе обновления?

Ответ: Увеличивает числовое значение поля на указанное число



Ответьте на вопрос



2.3 Какой файл должен присутствовать в папке, чтобы она считалась пакетом?



Ответьте на вопрос

2.3 Какой файл должен присутствовать в папке, чтобы она считалась пакетом?

Ответ: `__init__.py`



Выберите верный вариант ответа

2.4 Что делает конструкция `if __name__ == "__main__":`?

- a. Позволяет выполнять код только при прямом запуске
- b. Автоматически удаляет модуль после выполнения
- c. Запускается при любом импорте



Выберите верный вариант ответа

2.4 Что делает конструкция `if __name__ == "__main__":`?

- a. Позволяет выполнять код только при прямом запуске
- b. Автоматически удаляет модуль после выполнения
- c. Запускается при любом импорте



Выберите верный вариант ответа

2.5 Когда Python создаёт .рус файл?

- a. При запуске любого модуля
- b. Только при установке сторонних библиотек
- c. При первом импорте и если код изменился
- d. Только при запуске через `main.py`



Выберите верный вариант ответа

2.5 Когда Python создаёт .рус файл?

- a. При запуске любого модуля
- b. Только при установке сторонних библиотек
- c. При первом импорте и если код изменился
- d. Только при запуске через `main.py`



ВОПРОСЫ

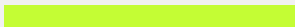




**Доработайте
практические
задания**



1. Список всех департаментов



Используя базу данных `hr`, выведите названия всех департаментов из таблицы `departments`. Каждое название должно отображаться с новой строки и иметь номер.

Пример вывода:

```
1. Administration
2. Marketing
3. Purchasing
...
27. Payroll
```

2. Сотрудники выбранного департамента

Добавьте к предыдущей программе возможность выбора департамента. Пусть пользователь введёт название из списка. Далее выведите всех сотрудников этого департамента, указав имя, фамилию и должность.

Пример вывода:

```
Enter department: Marketing
Michael Hartstein – Marketing Manager
Pat Fay – Marketing Representative
...
```



**Доработайте
практические
задания**



1. Поиск заказов с маленькой суммой

Прочитайте все документы из коллекции `orders`, у которых сумма (`amount`) **меньше 10**. Выведите каждый найденный заказ построчно.

Пример вывода:

```
{'_id': ObjectId('...'), 'id': 3, 'customer': 'Olga', 'product': 'Kiwi', 'amount': 9.6, 'city': 'Berlin'}
{'_id': ObjectId('...'), 'id': 5, 'customer': 'Olga', 'product': 'Banana', 'amount': 8, 'city': 'Madrid'}
```

2. Сохранение результатов в другую коллекцию

Сохраните все найденные заказы в новую коллекцию `orders_lesson_38`. После записи выведите, сколько документов было добавлено.

Пример вывода:

```
6 documents inserted into 'orders_lesson_38'.
```



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Список всех стран

Используя базу данных world, выведи **названия всех стран** из таблицы country. Каждое название должно отображаться с новой строки и иметь номер.

Пример вывода:

```
1. Aruba
2. Afghanistan
3. Angola
...
239. Zimbabwe
```

Домашнее задание

2. Города выбранной страны

Добавьте к предыдущей программе возможность выбора страны. Пользователь должен ввести название страны. Далее выведите **все города этой страны и их численность населения**.

Пример вывода:

Введите страну: Germany

Berlin – 3386667

Hamburg – 1704735

Munich [München] – 1194560

...



ВОПРОСЫ



Домашнее задание

1. Добавление товаров

Создайте программу, которая подключается к MongoDB и:

- выбирает базу `ich_edit` и коллекцию `products_<your_group>_<your_full_name>`
- очищает коллекцию перед началом
- добавляет 3 товара с полями: `name`, `price`, `stock`
- выводит сообщение о количестве добавленных товаров

Пример вывода:

3 products inserted.

Домашнее задание

2. Увеличение цен

Продолжите предыдущую задачу. Теперь программа должна:

- увеличить цену **всех товаров** на 20%
- вывести количество обновлённых записей
- затем вывести список всех товаров с новыми ценами

Пример вывода:

Prices updated for 3 products.

Updated products:

- Pen - \$1.80
- Notebook - \$4.79
- Backpack - \$30.00



ВОПРОСЫ



Заключение

